

Batman, Raw Gradient Loss Analysis

Yunru Zheng

February 22, 2026

1 Raw Gradient Loss Histogram

I drew a histogram of the raw gradient loss and played around a bit using different batch size and then I realized that I also need to take a snapshot at different stages of the optimization process since the graphs might look very different.

I'd like to start of saying that this is for b under the constraint. I might plot again for b unconstrained.

```
d_new = jnp.maximum(d_new, 0.1)
c_new = jnp.maximum(c_new, 0.5)
a_new = jnp.clip(a_new, 1.0, 3.0)

# Restrict b instead of a
max_b = d_new * (2.0 * a_new - 1.0)
b_new = jnp.clip(b_new, 0.0, max_b)
```

Figure 1: Constraint set 3.

I also plotted the raw mean and trimmed mean on the histograms. It doesn't look like they deviate too much from each other. However, in reality, that small difference determines the direction that the gradient descent is going, which is why the trimmed mean led to the previous error.

You can see in Figure 2 that the tail, especially the ones away from 0 are quite big, while after we reach the minimum loss, the tails do get smaller, but it becomes more problematic because the trimmed means now have opposite signs for 3 out of the 4 parameters. This explains the increase in loss that we saw in last few weeks. The final iteration has histograms similar to the best iteration.

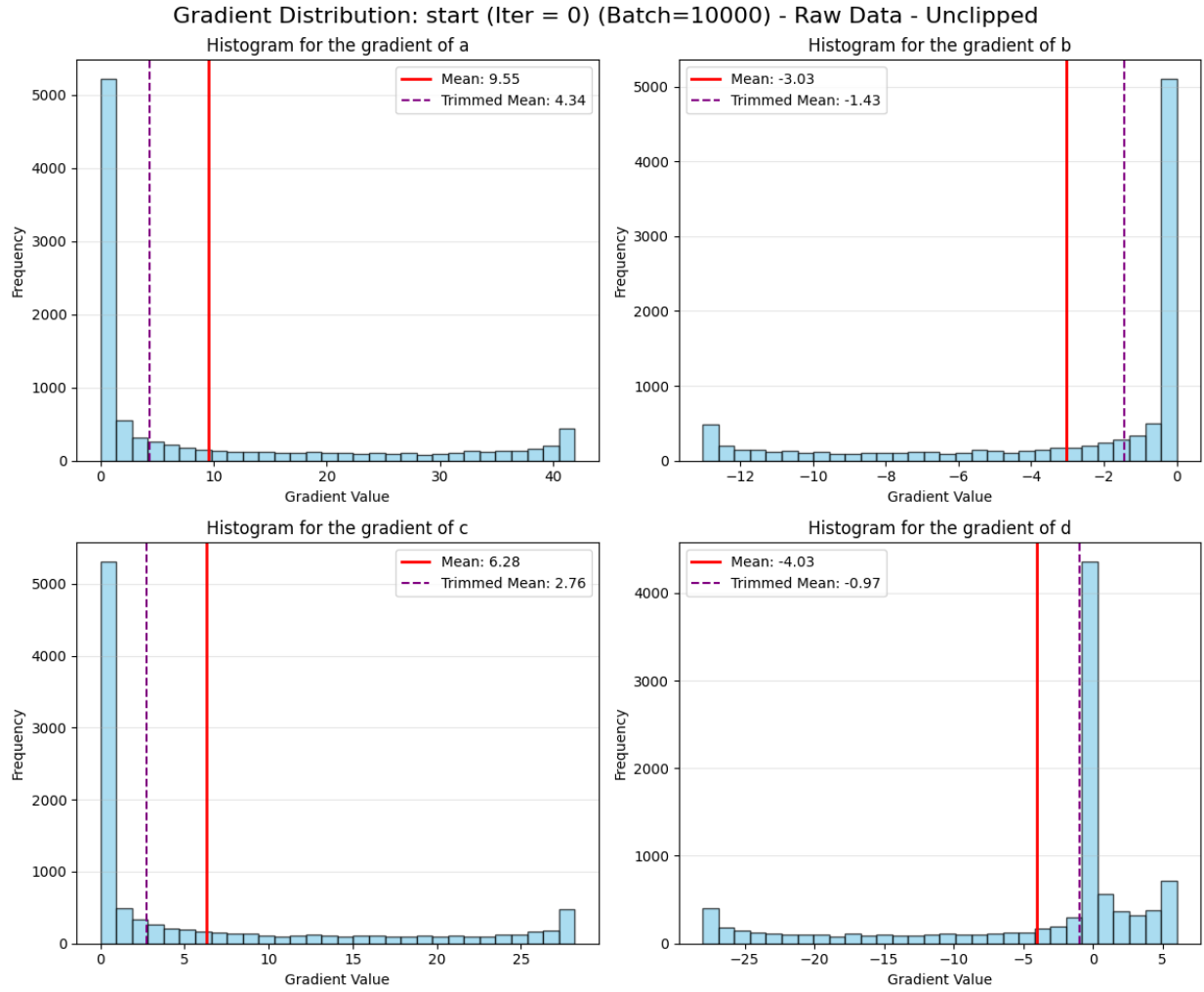


Figure 2: The histogram of the gradient of parameters with mean and trimmed mean at the beginning of the iteration.

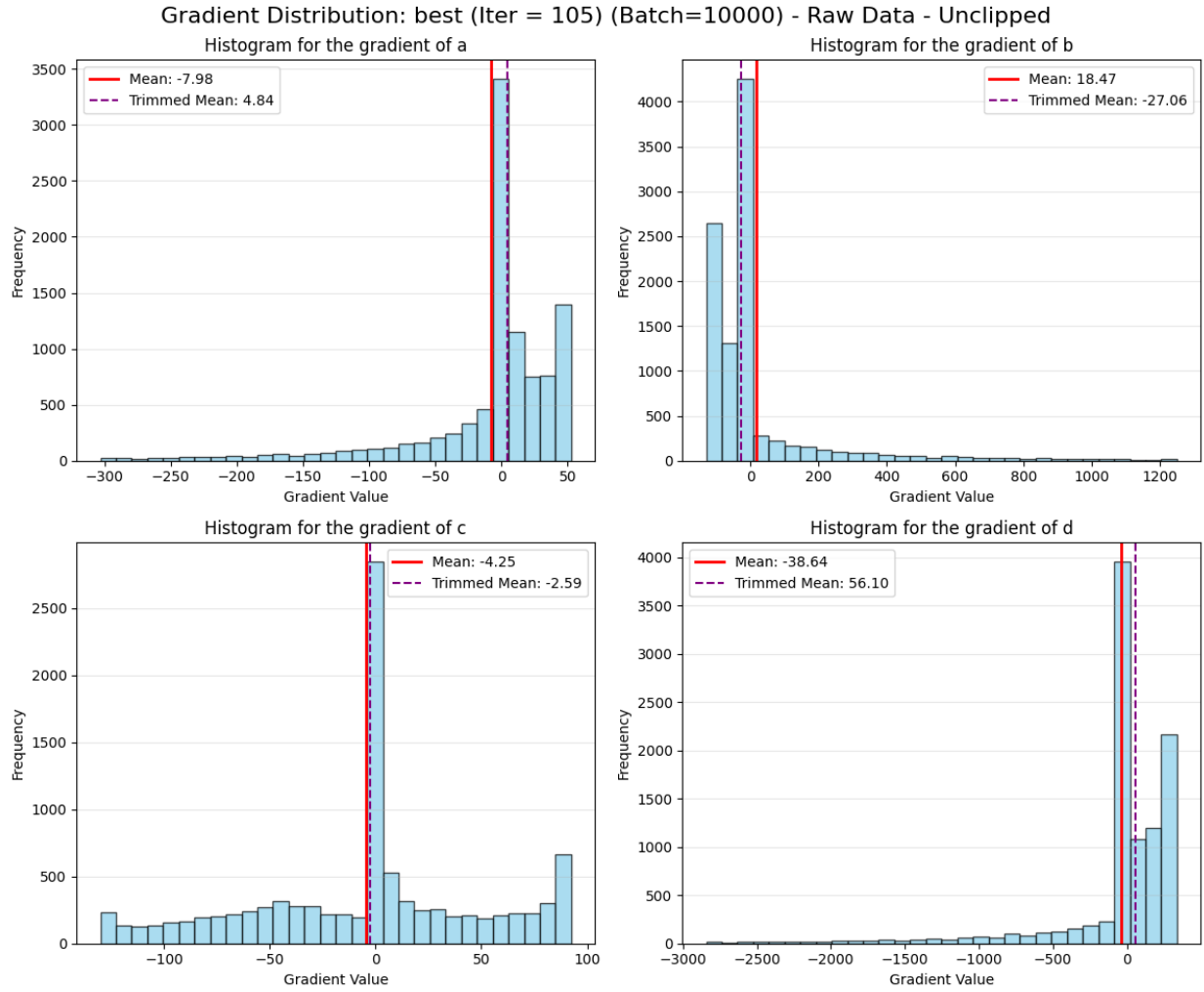


Figure 3: The histogram of the gradient of parameters with mean and trimmed mean at the iteration with minimum loss.

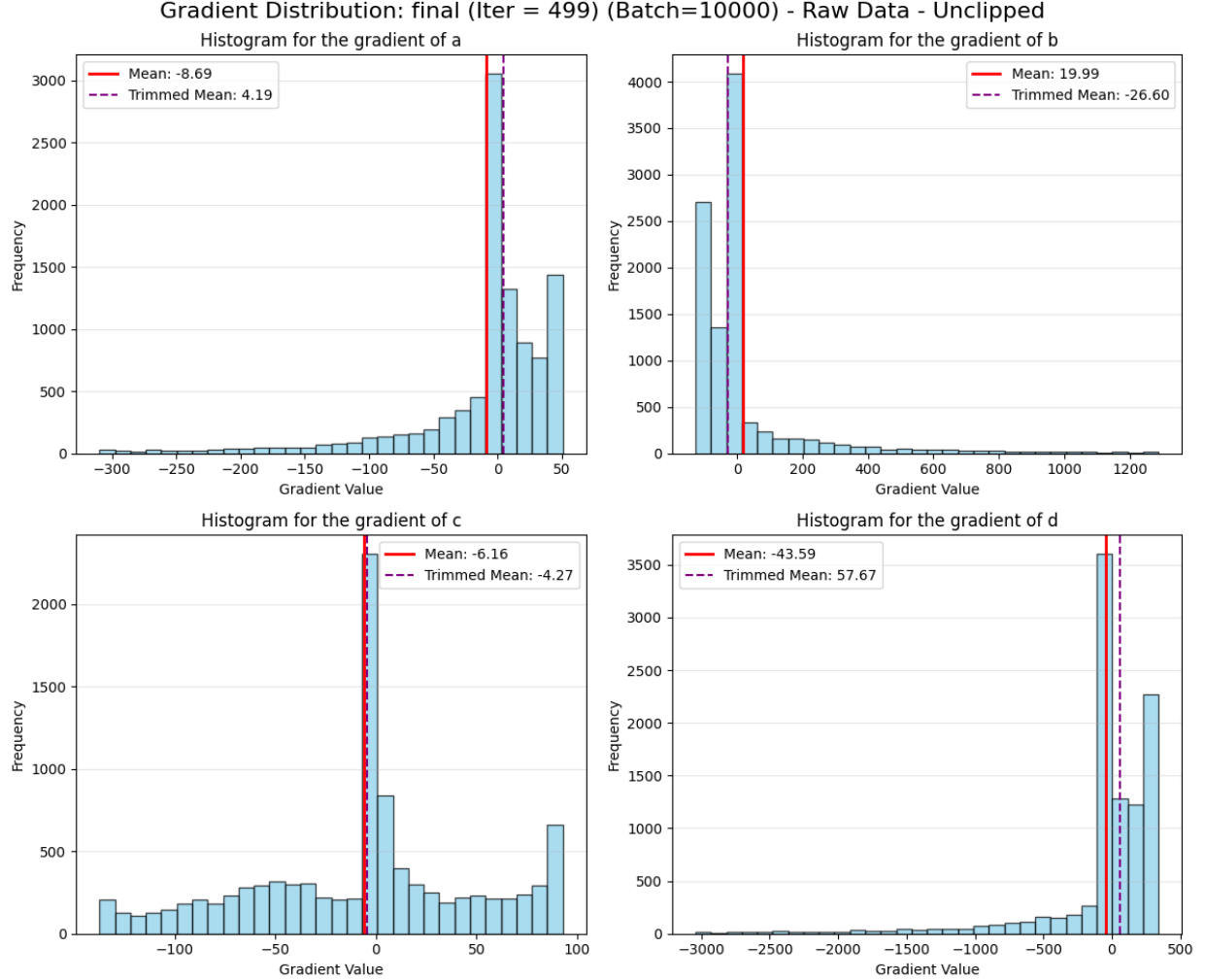


Figure 4: The histogram of the gradient of parameters with mean and trimmed mean at the end of the iteration.

Now that we tackled the trimming issue, I realize that my previous analysis on why constraints on parameters might not be completely valid. Under trimming, running without constraint would give negative parameters and neural nets that are not one-to-one. The constraints on c and d are still necessary, but I wonder if I could loosen the constraint on b .

The result is a bit confusing for me. In the constraint version, d is strictly decreasing until it hits my bound $d = 0.1$. However, while I didn't restrict d , it increases instead of decreases maybe due to its correlation with b . The parameters went to another direction and came back closer to the optimal value, although it's still not as optimal as the parameters with constraint. I'm also confused how rapidly the parameters fluctuate near iteration 1750. I picked the better one of the batman graph, but it's still not as good as having constraints on b .

```

Iter   0: Loss=21.4207 | Est=2.9647
      Theta: [1.9952244  0.5015133  0.9968593  0.20201649]
Iter  30: Loss=18.6624 | Est=3.0090
      Theta: [1.8283216  0.49665016  0.85172385  0.18694651]
Iter  60: Loss=13.0297 | Est=3.0073
      Theta: [1.6380857  0.22761714  0.5820106  0.1          ]
Iter  90: Loss=13.4563 | Est=3.0140
      Theta: [1.6381385  0.22762771  0.5428108  0.1          ]
Iter 120: Loss=13.9261 | Est=2.9948
      Theta: [1.6390761  0.22781523  0.53342426  0.1          ]
Iter 150: Loss=14.2903 | Est=2.9980
      Theta: [1.6441711  0.22883423  0.5308631  0.1          ]
Iter 180: Loss=14.2492 | Est=3.0022
      Theta: [1.648276  0.22965519  0.5284067  0.1          ]
Iter 210: Loss=14.1524 | Est=2.9761
      Theta: [1.6526513  0.23053026  0.5271238  0.1          ]
Iter 240: Loss=14.6306 | Est=2.9891
      Theta: [1.656535  0.23130701  0.52550393  0.1          ]
Iter 270: Loss=14.1040 | Est=3.0359
      Theta: [1.6601379  0.23202758  0.5240402  0.1          ]

Final Theta:, [1.6856723  0.22627242  0.5185705  0.12179695], learning rate: 0.0005, mixing ratio: 0.1
Optimal Theta, [1.6374176  0.22748351  0.5366698  0.1          ]

```

Figure 5: Data with constraint on b .

```

Iter   0: Loss=21.4207 | Est=2.9647
        Theta: [1.9952244  0.5015133  0.9968593  0.20201649]
Iter  30: Loss=17.9804 | Est=3.0108
        Theta: [1.8267683  0.58216935  0.84806925  0.18505178]
Iter  60: Loss=20.8488 | Est=3.0002
        Theta: [1.8029835  0.4202362  0.7547312  0.8147108]
Iter  90: Loss=20.3693 | Est=3.0404
        Theta: [1.7066941  0.4594587  0.7432711  0.79872644]
Iter 120: Loss=19.1675 | Est=2.9637
        Theta: [1.6197253  0.4971412  0.7321282  0.77936417]
Iter 150: Loss=18.9254 | Est=2.9956
        Theta: [1.5446345  0.5326544  0.72216254  0.7557837 ]
Iter 180: Loss=18.7050 | Est=3.0121
        Theta: [1.4835416  0.5655298  0.71437186  0.72682333]
Iter 210: Loss=17.9665 | Est=2.9508
        Theta: [1.4383534  0.59543043  0.7099464  0.6907678 ]
Iter 240: Loss=18.3512 | Est=3.0358
        Theta: [1.4088188  0.62326276  0.70990115  0.6439502 ]
Iter 270: Loss=17.5957 | Est=2.9876
        Theta: [1.3924174  0.652349  0.71514565  0.5766242 ]
Iter 300: Loss=16.7887 | Est=3.0201
        Theta: [1.3890289  0.69053245  0.7291593  0.4564344 ]
Iter 330: Loss=14.7741 | Est=3.0035
        Theta: [1.4628453  0.6912173  0.77757937  0.32723746]
Iter 360: Loss=14.3423 | Est=2.9972
        Theta: [1.5158366  0.66561294  0.77489394  0.29281065]
Iter 390: Loss=14.2382 | Est=3.0016
        Theta: [1.5432726  0.64326364  0.7451507  0.2755111 ]
Iter 420: Loss=14.1720 | Est=3.0178
        Theta: [1.5744207  0.6152713  0.71714044  0.27085683]
Iter 450: Loss=13.6718 | Est=2.9714
        Theta: [1.5984797  0.59369826  0.6873767  0.25405636]
Iter 480: Loss=15.5599 | Est=3.0341
        Theta: [1.6304272  0.5596019  0.662421  0.26573142]
Iter 1000: Loss=13.5362 | Est=3.0159
        Theta: [1.71737  0.45840403  0.5546467  0.22673133]

Final Theta:, [1.6719708  0.43476054  0.57335037  0.21262118], learning rate: 0.0005, mixing ratio: 0.1
Optimal Theta, [1.6252145  0.5695038  0.664703  0.24506953]

```

Figure 6: Data without constraint on b .

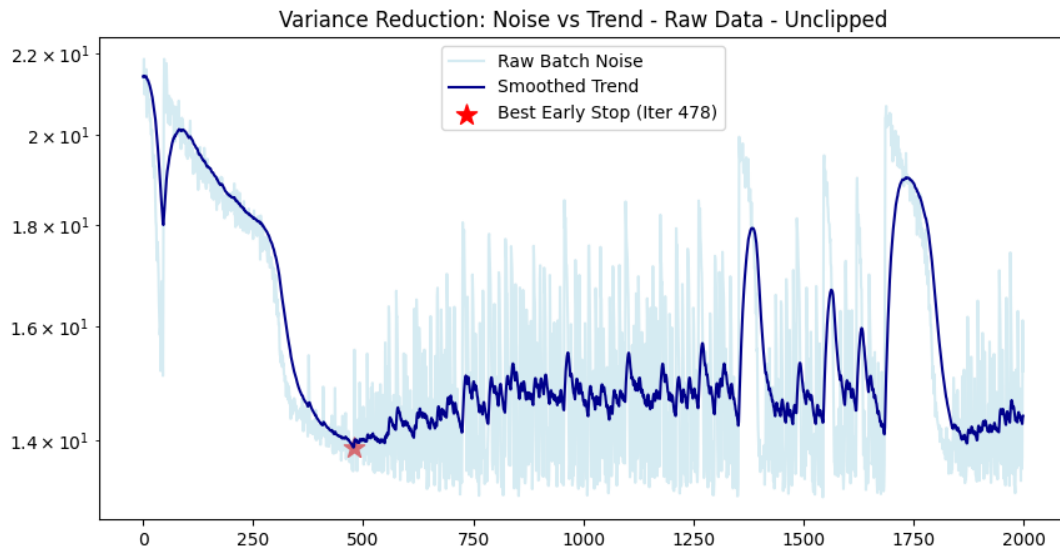


Figure 7: Loss without constraint on b .

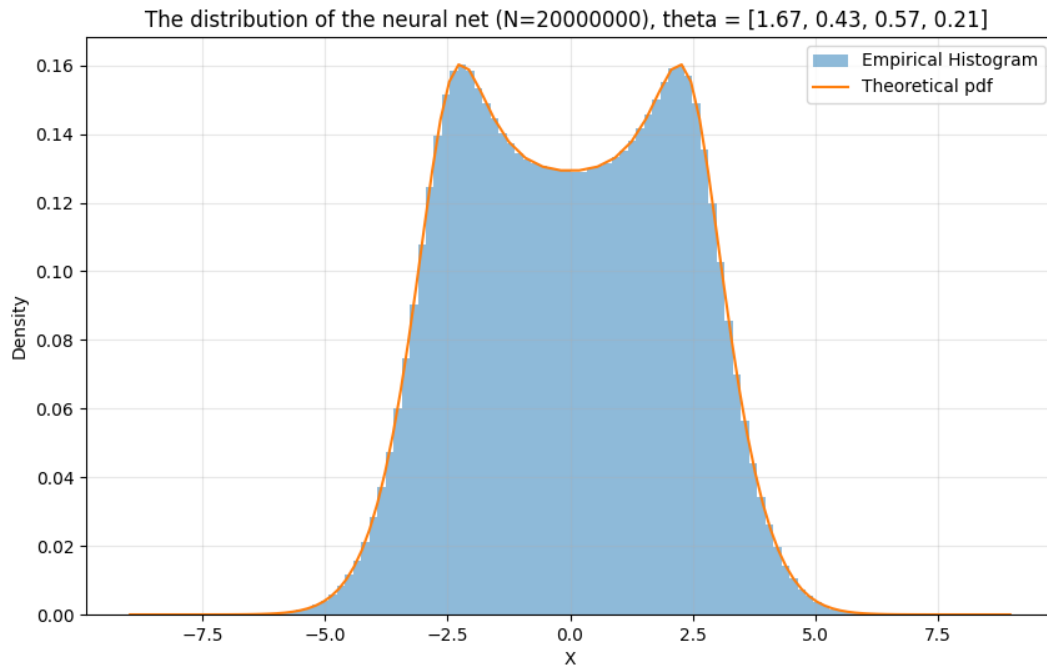


Figure 8: Batman with paramters [1.67, 0.43, 0.57, 0.21].